# ProveML: Inline Claim Markup for Deterministic Verification of AI-Generated Text

Shane Deconinck
trustedagentic.ai
shane@trustedagentic.ai

March 2026

**Abstract**

Organizations are deploying LLMs in consequential settings under regulatory pressure, but most generated text carries no machine-checkable link between individual claims and the underlying data. Each generation of models produces more fluent output, but none can reliably signal its own uncertainty.

We present ProveML, a lightweight Markdown extension that places AI-generated claims inside a verifiable boundary. Three inline constructs declare entities, link facts to a data source, and check qualitative judgments against composable thresholds. Verification is deterministic: facts resolve by key-value lookup, thresholds by arithmetic comparison, with no model in the loop. Because verification is instant, it can sit inside a generation loop, feeding errors back to the LLM to improve consistency with the data.

Across four models (3.8B to API-scale, 3 runs each), we observe a three-regime pattern: below a capability threshold, models fail to produce verifiable markup reliably; in an intermediate regime, models emit markup stably but make addressability errors the loop can selectively repair; above threshold, ProveML serves as a compliance layer. A cross-domain evaluation on both a synthetic educational dataset and real SEC EDGAR financial data shows the regime structure is benchmark-dependent: all four models enter the verifiable boundary when context complexity is low. We provide the formal grammar, verification algorithm, correctness argument, and evaluation across four experiments spanning two domains and four models.

**Keywords:** AI verification, markup language, deterministic verification, structured data, threshold registry, hallucination detection, inline tagging

## 1 Introduction

Large Language Models (LLMs) produce fluent, contextually appropriate text that is increasingly difficult to distinguish from human-authored content. This fluency, however, coexists with a fundamental reliability problem: **LLMs hallucinate**. They generate text that is plausible but factually incorrect, including fabricated citations, invented statistics, and misattributed claims.

Recent theoretical work suggests that **hallucination is not merely an incidental bug but reflects structural properties** of how language models operate. Kalai and Vempala (2024) prove formally that any language model satisfying natural calibration conditions *must* hallucinate, at a rate bounded by the number of facts appearing rarely in training data; subsequent work Kalai et al. (2025) shows that standard training and evaluation pipelines actively reward guessing over acknowledging uncertainty. Xu et al. (2025) use computability theory to show that LLMs cannot learn all computable functions and will therefore inevitably produce confabulations

when used as general-purpose reasoners. More broadly, hallucination is a well-documented reliability problem across deep-learning-based natural language generation systems Ji et al. (2023).

**Current mitigations reduce hallucination rates but cannot eliminate them.** RLHF improves instruction-following and truthfulness Ouyang et al. (2022). Even so, aligned models may still produce confident-sounding but incorrect responses. RAG grounds responses in documents but models can still misrepresent retrieved context; Magesh et al. (2025) found hallucination rates exceeding 17% in both Lexis+ AI and Westlaw AI-Assisted Research, despite vendor claims of near-elimination. Emerging interpretability research can detect some internal uncertainty signals Orgad et al. (2025); Farquhar et al. (2024), but probes do not generalise across tasks, entropy methods fail on confident errors, and all require white-box access. McCain et al. (2026) report that 73% of agent tool calls still involve human oversight, finding that agents systematically overestimate their own success rates.

**Meanwhile, the consequences are real and adoption is accelerating.** In 2025, Deloitte delivered a government report with AI-fabricated citations Rudge (2025). Earlier, fabricated ChatGPT citations were submitted to a U.S. federal court U.S. District Court, S.D.N.Y. (2023), and Google's first Bard demo hallucinated on camera, contributing to a $100 billion market cap drop Elias (2023). Yet the majority of organisations now use AI in at least one function McKinsey & Company (2025), and Gartner projected over 80% of enterprises would deploy generative AI by 2026 Gartner (2023). **Organisations are not waiting for hallucination to be solved**; they are deploying now and managing the risk.

This creates precisely the conditions described in Bainbridge's classic analysis of the **ironies of automation** Bainbridge (1983) and well-documented in the trust-in-automation literature Lee and See (2004); Parasuraman and Riley (1997): as automated systems become more reliable, human operators trust them more and monitor them less. When the rare failure occurs, it is more likely to go undetected precisely because the operator has become complacent. In the context of AI-generated text, this dynamic is particularly acute. An LLM that produces correct output 97% of the time creates a more dangerous situation than one that produces correct output 80% of the time, because the 97% system induces trust that suppresses human verification behaviour.

The regulatory landscape compounds the problem. The EU AI Act European Parliament (2024) classifies AI systems used for consequential decisions as "high-risk" (Annex III), requiring transparency, human oversight, and risk management. Yet to our knowledge, **no existing framework combines inline claim markup, deterministic mismatch detection against structured data, and composable threshold inference** in a single system.

Current approaches to the AI reliability problem include several strategies, none of which addresses the structural verification gap:

1. **Post-hoc fact-checking** (FActScore Min et al. (2023), SAFE Wei et al. (2024), FacTool Chern et al. (2023), SelfCheckGPT Manakul et al. (2023), RefChecker Hu et al. (2024)): decompose, sample, or re-check generated text after the fact against external sources or model-generated consistency signals. These systems use non-deterministic checks in the verification loop, making verification itself probabilistic and subject to many of the same failure modes as the generation process.

2. **Output validation** (Guardrails AI Guardrails AI (2024), DSPy Khattab et al. (2024), NeMo Guardrails NVIDIA (2023)): programmatic constraint enforcement that can operate at output or sentence level. Guardrails AI's provenance validators support sentence-level checks against source text; DSPy compiles declarative LM pipelines into optimized prompts. However, verification in these systems still relies on LLM judgments or embedding similarity, not deterministic checks against structured data stores with addressable paths.

3. **Attribution** (RARR Gao et al. (2023), WebGPT Nakano et al. (2021), OpenAI response annotations): link claims to source documents or passages. Some systems (e.g., Google's grounding API Google (2024)) additionally return support scores and claim-to-source metadata. These track *where* a claim came from, not *whether the claimed value is correct*. Verification, where present, is probabilistic and against unstructured text.

**None of these provide a structural guarantee** that every factual claim in AI-generated text is an addressable, deterministically verifiable reference to a source record in a structured data store.

We present ProveML (Provenance Markup Language), a system that fills this gap. ProveML is an inline claim markup language embedded in AI-generated text where:

- Every entity reference is verified against a structured data store (name match).
- Every factual claim resolves to a specific addressable path in the data store (value match).
- Every inference is checked against a threshold registry defined outside the generation process.
- Verification is fully deterministic, with no model in the verification loop.
- Unverified or mismatched claims are structurally detectable and can be flagged by the rendering layer.

The closest structural analog is iXBRL (Inline XBRL) XBRL International (2013), which embeds machine-readable tags in human-readable financial reports, enabling automated audit of reported figures. ProveML extends this concept from human-authored financial text to AI-generated text across any domain, adding inference chains, entity scoping, and threshold registries (see Figure 6 for the architectural distinction).

The key insight underlying ProveML is that *verification should be a structural property of the generated text, not a post-hoc evaluation.* Just as a type system prevents certain classes of errors at compile time rather than detecting them at runtime, ProveML makes certain classes of AI errors (factual inaccuracy, entity misattribution, unjustified inference) structurally detectable at generation time.

**Design philosophy.** ProveML is deliberately assembled from established patterns rather than invented from scratch:

- Host language: Markdown (natively produced by LLMs)
- Inline tagging model from iXBRL XBRL International (2013)
- Operator vocabulary from FHIR clinical reference ranges HL7 International (2023) and JSON Schema validation
- Fact store from the Entity-Attribute-Value (EAV) pattern
- Verify-then-render pipeline from standard compiler design

A sceptical reader should find each component familiar. What ProveML contributes is the *specific combination* applied to AI-generated text, together with two mechanisms we have not seen elsewhere: the threshold registry as a vocabulary constraint (the generating process cannot assert what has not been defined), and the verify-fix loop operating at claim-level granularity.

The contributions of this paper are:

1. **A formal specification** of the ProveML markup language, including grammar, verification algorithm, and a correctness argument (Sections 3–6).

2. **A threshold registry** of composable inference primitives. Domain experts define the qualitative judgments that can be verified; in regulated deployments, the prompting layer can enforce that unverifiable observations use exploratory language.

3. **An empirical evaluation** across four models (3.8B to API-scale) revealing a three-regime capability pattern. Most first-pass failures are addressability errors, not value mismatches (Section 7).

4. **A reference implementation** with 156 tests (plugin, grammar, paper-example, CLI, and detection checks), tested against both a synthetic educational dataset and real SEC EDGAR financial data (Section 9).

# 2 Architecture

Figure 1 contrasts the standard approach (data in, text out, no verification) with ProveML's pipeline.

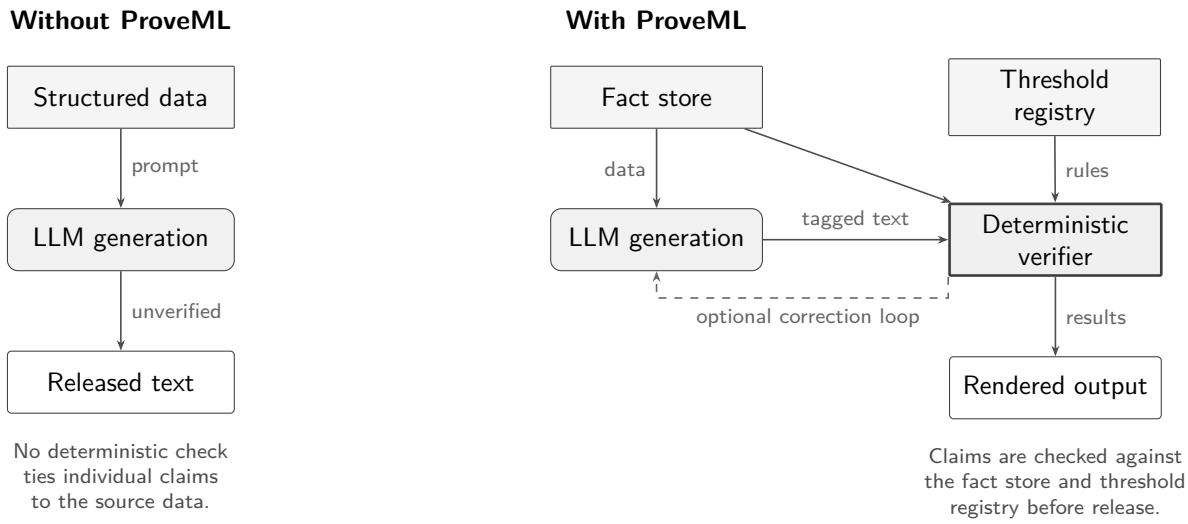


Figure 1: Without ProveML (left): data goes in, text comes out, no verification. With ProveML (right): every claim is checked against the data source and threshold registry, with an optional correction loop. Rounded boxes denote generative components; squared boxes denote data stores and deterministic processing.

## 2.1 Fact Store

The fact store is a flat key-value index following the Entity-Attribute-Value (EAV) pattern. Any source with records and fields can be flattened into it. The flattening is mechanical: each record becomes `type:id.field` → `value`, with optional sub-paths for nested data.

**Definition 1** (Fact Store Path)**.** *A fact store path has the form:*

$$\textit{entity_type} : \textit{entity_id}\,(\,.\ \textit{sub_type} : \textit{sub_id})^{*}\ .\ \textit{field_name} \rightarrow \textit{value}$$

*where the sub-path is optional and repeatable for nested data. The verifier treats terminal values as atomic facts with canonical string representations; threshold operators may impose additional type requirements. For example,* `account:901.transaction:2026-01.amount` *addresses a specific transaction's amount.*

Examples (flat fields and nested sub-paths):

```
account:901.holder                        -> "Acme Corp"
account:901.balance                       -> -12400
account:901.balance._unit                 -> "EUR"
account:901.overdue                       -> 3
account:901.transaction:2026-01.amount    -> 5200
account:901.transaction:2026-01.amount._unit -> "EUR"
```

ProveML separates two verification modes. Direct fact references use exact string equality against the store's canonical representation: if the store canonicalizes `18.50` to `18.5`, the claim must say `18.5`. Threshold predicates use typed comparison (numeric, set membership, etc.). The fact store is the canonicalization point: any consistent flattening scheme works, provided the same representation is used for both storage and generation.

**Units.** Fact store entries may carry an optional unit via a companion key:

```
account:901.balance        -> -12400
account:901.balance._unit  -> "EUR"
patient:308.glucose        -> 142
patient:308.glucose._unit  -> "mg/dL"
```

Units are modeled structurally in the fact store via companion metadata (e.g., `balance` and `balance._unit`), but direct fact references verify the canonical surface form shown to the reader (e.g., `%[balance]{-12400 EUR}`). When no unit is present, value-only comparison applies. Threshold definitions may declare an expected unit; if the fact store's unit for that field does not match, verification fails. The grammar is unchanged: unit enforcement is a property of the data model, not the markup syntax.

This follows iXBRL's principle that numeric facts should be self-describing, while keeping ProveML's verification mechanism minimal. In regulated deployments, unit strings may need to resolve to canonical identifiers from standardised vocabularies (e.g., UCUM for clinical units, ISO 4217 for currencies); this is a natural extension but beyond the scope of the present specification.

**Data sources.** This specification defines verification against a single flat store. The generating model verifies against whatever the deployer places in the store; source selection is a deployment decision. Multiple stores, explicit source provenance per fact, or a source prefix on entity paths are natural extensions but beyond the current scope.

ProveML does not verify the fact store itself. The system's guarantee is "consistent with the data source", not "true." If the source data is wrong, verified claims will be wrong. The fact store is the single point of trust.

**Immutability**: Verification is always relative to a particular fact-store state. During a verification session, the fact store must remain immutable. If the underlying data changes, previously verified texts are not assumed to remain valid and must be re-verified. Deployments that require audit trails can bind a snapshot identifier (hash or timestamp) to each verification result; the reference implementation supports this as an optional parameter. In the evaluation, the fact store is loaded once per session and not modified, so immutability holds trivially without explicit snapshot binding.

## 2.2 Threshold Registry

The threshold registry draws on the design patterns of clinical reference ranges (HL7 FHIR HL7 International (2023)), JSON Schema validation, and monitoring alert systems (Grafana, Datadog). The common insight across these systems: domain experts think in *named ranges with semantic labels*, not in operator expressions. A clinician defines "normal glucose = 70–100 mg/dL", not "glucose >= 70 AND glucose <= 100".

**Definition 2** (Threshold)**.** *A threshold is a named predicate with a semantic label:*

$$T = (\mathit{name}, \mathit{field}, \mathit{predicate}, \mathit{unit?}, \mathit{label}, \mathit{source})$$

*where predicate is a tagged union over the operator forms in Table 1:*

$$\begin{aligned} \mathit{predicate} ::= &\ (\texttt{lt} \mid \texttt{gt} \mid \texttt{lte} \mid \texttt{gte} \mid \texttt{eq} \mid \texttt{neq},\ \mathit{value}) \\ \mid &\ (\texttt{between},\ \mathit{low},\ \mathit{high}) \\ \mid &\ (\texttt{in},\ \mathit{values}^{+}) \\ \mid &\ (\texttt{is_null}) \\ \mid &\ (\texttt{diff_gt},\ \mathit{value},\ \mathit{direction}) \end{aligned}$$

*where value, low, high are numbers and direction is one of* `higher`*,* `lower`*, or* `either`*. The optional unit constrains the field's companion* `._unit` *value in the fact store; it does not alter the operator vocabulary.*

| Operator | Display label | Example |
|---|---|---|
| *Core (numeric comparison)* | | |
| `lt` | is less than | `price lt 10` |
| `lte` | is at most | `rating lte 3` |
| `gt` | is greater than | `stock gt 0` |
| `gte` | is at least | `score gte 75` |
| `eq` | equals | `status eq "active"` |
| `neq` | is not | `status neq "archived"` |
| `between` | is between | `score between 25 50` |
| *Extended* | | |
| `in` | is one of | `category in {A,B,C}` |
| `is_null` | is missing | `value is_null` |
| `diff_gt` | differs by more than | `score diff_gt 20` |

Table 1: Threshold operator vocabulary. Core operators cover numeric comparison and ranges. Extended operators handle categorical data, missing values, and cross-entity comparison.

Examples of threshold definitions:

```
IS_LOW:        score lt 25         "critically low"
IS_MODERATE:   score between 25 50 "moderate"
IS_ACTIVE:     status in {A,B,C}  "active"
IS_MISSING:    value is_null       "no data"
MUCH_HIGHER:   score diff_gt 20    "much higher"
IS_NEGATIVE:   balance lt 0 [EUR]  "negative balance"
```

Threshold labels are field-specific, not global semantics: a range labeled "moderate" for one variable need not have the same meaning for another. The *source* field in each definition records where the threshold was defined, supporting audit.

**Design rationale.** The operator set is deliberately small. Logical composition (`AND`, `OR`, `NOT`) is handled at the inference level, not the threshold level, keeping individual thresholds simple enough for non-technical domain experts to define and audit. Complex patterns emerge from combining simple thresholds:

```
?[low: IS_LOW]{critically low score}
?[missing: IS_MISSING]{no recent data}
?[risk: @low AND @missing]{low score with missing data}
```

**Cross-entity comparison (`diff_gt`).** The `diff_gt` operator requires two entity values. The verifier resolves these using the explicit path syntax: `THRESHOLD_NAME(entity:id.field)`. The first operand is the current entity context; the second is the explicit path argument. For example:

```
?[gap: MUCH_HIGHER(region:US.sales)]{much higher}
```

computes $|current_entity.\texttt{sales} - \texttt{region:US.sales}|$ and checks whether it exceeds 20 in the specified direction. If no explicit path is provided, `diff_gt` is unverifiable and the verifier returns an error.

The AI may only reference thresholds from the registry. It cannot invent comparison values, qualitative magnitudes, or directional claims. This is the primary defense against threshold manipulation: the qualifier itself becomes a verifiable claim.

# 3 Syntax

ProveML extends Markdown with three constructs, using characters (`@`, `%`, `?`) that do not conflict with standard Markdown syntax. The syntax draws on conventions from popular Markdown extensions: bracketed spans (Pandoc, Djot), pipe-separated parameters (MediaWiki, Obsidian), and quoted metadata in references (Markdown link titles). The choice of Markdown is deliberate: LLMs produce it fluently and existing renderers pass ProveML constructs through unchanged.

A complete example before the formal definitions:

```
(* Simple form: entity, then facts follow *)
@[sensor:42]{Sensor X} reads %[value]{29.99} with %[stock]{150} in stock.

(* Scoped form: facts inside entity braces *)
@[sensor:42 "Sensor X"]{reads %[value]{29.99}, %[stock]{150} in stock}

(* Nested scopes: outer entity with multiple inner entities *)
@[facility:7 "Plant North"]{hosts
  @[sensor:42 "Sensor X"]{reading %[value]{29.99}} and
  @[sensor:43 "Sensor Y"]{reading %[value]{18.50}},
  with %[sensorCount]{12} sensors total}
```

The simple form uses linear carry-forward; the scoped form uses lexical binding. In the nested example, each `value` binds to its enclosing sensor, while `sensorCount` binds to `facility:7` (outer scope). The verifier maintains a scope stack, so binding is always deterministic.

## 3.1 Entity Reference

Two forms, fully backward compatible:

```
@[entity_type:entity_id]{display_name}
@[entity_type:entity_id "verified_name"]{prose with facts}
```

**Simple form** `@[sensor:42]{Sensor X}`: the brace content serves as both the display text and the verified name. The verifier checks `fact_store`[`sensor:42.name`] = `"Sensor X"`. If the name does not match, verification fails.

**Scoped form** `@[sensor:42 "Sensor X"]{prose...}`: the quoted string is verified against the fact store; the braces contain prose where fact references bind explicitly to this entity. The scoped form is needed when carry-forward would give the wrong binding: multi-entity comparisons, nested entities, or languages where the subject follows the predicate:

```
@[sensor:42 "Sensor X"]{costs %[price]{29.99}} vs
@[sensor:43 "Sensor Y"]{costs %[price]{18.5}}
```

Only ProveML constructs (`@[...]`, `%[...]`, `?[...]`) inside the braces are verified; surrounding prose is not. A scoped entity does not confer verification status on its prose. It is up to the rendering engine to visually distinguish verified constructs from unverified prose, and to indicate which entity each fact binds to (for example, through color coding or scope indicators).

### 3.2 Fact Reference

```
%[field_name]{value}
```

A verifiable factual claim. Binding follows two rules:

1. **Lexical scope**: a fact inside `@[entity "name"]{...}` braces binds to that entity.
2. **Linear fallback**: a fact outside any scoped entity braces binds to the nearest preceding `@[entity]`.

Rule 1 takes priority. The simple case (`@[e]{Name} ... %[f]{v}`) works as before. The scoped form handles non-English word order, tables, and interleaved comparisons without introducing explicit paths on facts; binding is always structural, never controlled by the LLM.

**Verification**: The verifier checks that `fact_store`[*bound_entity*.*field*] = *value*.

Three outcomes: *verified* (match), *mismatch* (value differs), *unverifiable* (field or entity not found).

### 3.3 Inference Reference

```
?[label: condition]{display_text}
```

A verifiable logical observation. Conditions reference the threshold registry or combine prior inference labels.

**Definition 3** (Condition Grammar)**.**

$$
\begin{aligned}
\mathit{condition} &::= \mathit{or_expr} \\
\mathit{or_expr} &::= \mathit{and_expr}\ (\texttt{OR}\ \mathit{and_expr})^* \\
\mathit{and_expr} &::= \mathit{unary}\ (\texttt{AND}\ \mathit{unary})^* \\
\mathit{unary} &::= \texttt{NOT}\ \mathit{unary} \mid \mathit{primary} \\
\mathit{primary} &::= \mathit{THRESHOLD_NAME} \mid \mathit{THRESHOLD_NAME}\texttt{(}\mathit{path}\texttt{)} \mid \texttt{@}\mathit{label}
\end{aligned}
$$

### 3.4 Formal Grammar

A simplified EBNF for ProveML constructs embedded in Markdown:

Listing 1: ProveML Grammar (EBNF)

```
(* ProveML constructs inside Markdown inline content *)
entity_ref_simple = "@[" , entity_path , "]{" , display_text , "}" ;
entity_ref_scoped = "@[" , entity_path , " " , DQUOTE ,
                    entity_name , DQUOTE , "]{" ,
                    scoped_content , "}" ;
entity_ref        = entity_ref_simple | entity_ref_scoped ;

fact_ref          = "%[" , field_name , "]{" , value_text , "}" ;
inference_ref     = "?[" , label , ": " , condition , "]{" ,
                    display_text , "}" ;

(* Paths and identifiers *)
entity_path       = identifier , ":" , entity_id ;
entity_id         = ( letter | digit ) ,
                    { letter | digit | "_" | "-" } ;

field_name        = field_segment , { "." , field_segment } ;
field_segment     = meta_field | path_field | identifier ;
meta_field        = "_" , identifier ;
path_field        = identifier , ":" , entity_id ;

label             = identifier ;
identifier        = letter , { letter | digit | "_" } ;

(* Text content *)
entity_name       = { char - '"' } ;
display_text      = { char - "}" } ;
scoped_content    = { char } ;   (* parsed recursively *)
value_text        = { char - "}" } ;

(* Condition grammar *)
condition         = or_expr ;
or_expr           = and_expr , { " OR " , and_expr } ;
and_expr          = unary_expr , { " AND " , unary_expr } ;
unary_expr        = [ "NOT " ] , primary ;
primary           = threshold_name
                  | threshold_name , "(" , path , ")"
                  | "@" , label ;

(* Terminals *)
threshold_name    = uppercase , { uppercase | "_" } ;
path              = entity_path , "." , field_name ;
DQUOTE            = '"' ;
letter            = "a"-"z" | "A"-"Z" ;
uppercase         = "A"-"Z" ;
digit             = "0"-"9" ;
```

This grammar specifies ProveML constructs only; surrounding paragraph, list, table, and block structure is delegated to the host Markdown parser.

# 4 Verification Algorithm

Figure 2 illustrates the decision logic for each ProveML construct.

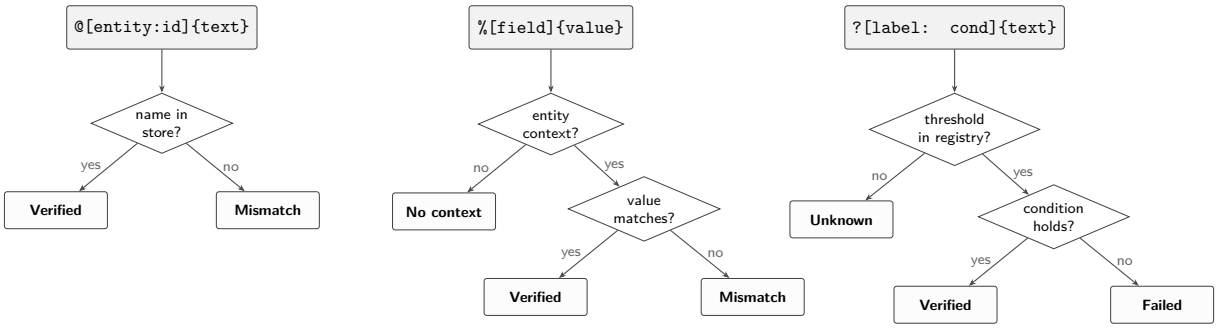


Figure 2: Verification decision flow for each ProveML construct type. Every path terminates in a deterministic outcome with no probabilistic judgment.

**Algorithm 1** ProveML Verification

**Require:** ProveML document $D$, fact store $F$, threshold registry $R$, optional snapshot $s$
**Ensure:** Verification result $V$

initialize $V$, $\mathit{labels} \leftarrow \emptyset$, $\mathit{entityStack} \leftarrow []$, $\mathit{currentEntity} \leftarrow$ `null`
**if** $s$ is provided **then**
  $V.\mathit{snapshot} \leftarrow s$
**end if**
**for** each token $t$ in $\textsc{Tokenize}(D)$ **do**
  **if** $t.\mathit{type} =$ `entity` **then**
    $p \leftarrow t.\mathit{entityType}{:}t.\mathit{entityId}$
    $V$.add($\textsc{VerifyEntity}(t, p, F)$)
    update $\mathit{currentEntity}$ and $\mathit{entityStack}$ from $t$ ▷ scoped or carry-forward
  **else if** $t.\mathit{type} =$ `entity_close` **then**
    restore $\mathit{currentEntity}$ from $\mathit{entityStack}$ if possible
  **else if** $t.\mathit{type} =$ `fact` **then**
    $V$.add($\textsc{VerifyFact}(t, \mathit{currentEntity}, F)$)
  **else if** $t.\mathit{type} =$ `inference` **then**
    $r \leftarrow \textsc{Evaluate}(t.\mathit{condition}, \mathit{currentEntity}, F, R, \mathit{labels})$
    $\mathit{labels}[t.\mathit{label}] \leftarrow r$
    $V$.add($r$)
  **end if**
**end for**
**return** $V$

VerifyEntity returns Verified, EntityNotFound, or NameMismatch. VerifyFact resolves the current entity path, appends optional `._unit` metadata to the expected surface form, and compares by string equality; absence of context or fields yields NoContext or FieldNotFound. Evaluate handles registered thresholds, label references, and boolean composition. In the implementation, $V$.add($\cdot$) updates both the total claim count and the verified count when the returned result is verified.

**Property 1** (Time Complexity)**.** *Verification is $O(n)$ where $n$ is the number of ProveML constructs, assuming $O(1)$ fact store lookup. The fact store is a hash map keyed by the full path string (e.g.,* `sensor:42.price`*), so this assumption holds in practice.*

**Property 2** (Decidability)**.** *Verification is trivially decidable. All conditions reduce to ground propositional logic over fully instantiated boolean expressions. There are no quantifiers, no recursion, and no unbounded computation.*

**Comparison semantics.** Fact verification uses **string equality**: the claimed value and the fact store value are both converted to their string representations and compared exactly. This means `%[price]{29.99}` matches a fact store value of `29.99` but *not* `29.990` or `30`. Threshold evaluation uses **numeric comparison**: both operands are parsed as numbers before applying the operator. Implementations must define a canonical string form for fact store values (the reference implementation uses JavaScript's `String()` coercion, which drops trailing zeros: `18.50` becomes `"18.5"`). This canonicalization is implementation-defined; a Python or Rust implementation may produce different representations for the same numeric value. Portability requires that the fact store and verifier share the same canonicalization rule, and that the rule is documented per deployment.

## 5 Verification Correctness

**Property 3** (Claim-Store Consistency)**.** *If the ProveML verifier reports* VERIFIED *for a claim, the claimed value equals the corresponding value in the fact store (for entities and facts) or the threshold condition holds (for inferences).*

**Proof sketch.** By case analysis over construct types. Each reduces to a single deterministic check with no external dependencies:

- **Entity reference** `@[type:id]{text}`: VERIFIED iff $F[\texttt{type:id.name}] = \textit{text}$. String equality.
- **Fact reference** `%[field]{value}`: VERIFIED iff $F[\textit{entity}.\textit{field}] = \textit{value}$. String equality against the current entity context, which the algorithm maintains as state updated on each entity declaration.
- **Threshold inference** `?[l: T]{text}`: VERIFIED iff $\textit{compare}(F[\textit{entity}.T.\textit{field}], T.\textit{op}, T.\textit{value})$. Standard arithmetic comparison.
- **Label reference** `@label`: VERIFIED iff the referenced label was previously verified.
- **Connectives** `AND`, `OR`, `NOT`: Standard propositional semantics over previously established verification results.

The simplicity is intentional. The verification guarantee should be obvious to anyone reading the algorithm, not because it is trivial, but because the system was designed so that correctness requires no complex reasoning. The complexity of ProveML lives in the *design* (what to verify, how to constrain the AI), not in the verification logic itself.

**Assumptions.** Soundness requires:

1. The fact store faithfully represents the source data at snapshot time.
2. The threshold registry contains meaningful thresholds defined by domain experts.
3. The fact store is immutable during verification (snapshot semantics).

**Entity scoping.** Fact binding uses lexical scope: a fact inside a scoped entity's braces binds to that entity. A fact outside any scoped braces binds to the nearest preceding entity (linear fallback). Nested scoped entities resolve inward.

## 6 Rendering

ProveML specifies four verification states. The rendering must visually distinguish them; the choice of colors and styles is left to the implementation.

| State | Indicator | Meaning |
|---|---|---|
| Verified fact | ✓ | Claimed value matches fact store |
| Verified inference | ✓ | Threshold condition holds |
| Mismatch | × | Claimed value differs from fact store |
| Unverifiable | ? | Field or entity not found |

Table 2: ProveML verification states. Visual rendering (colors, underline styles) is implementation-defined.

Three reading modes support progressive disclosure: *Read* (minimal indicators), *Verify* (full status colors with tooltips), and *Audit* (proof paths visible inline). Hovering over any entity or fact highlights all claims bound to the same entity, making the scope binding visible without inspecting markup. Figures 3–5 show three rendering states using SEC EDGAR financial data; a live version is available at `demo/proveml-demo.html` in the repository.

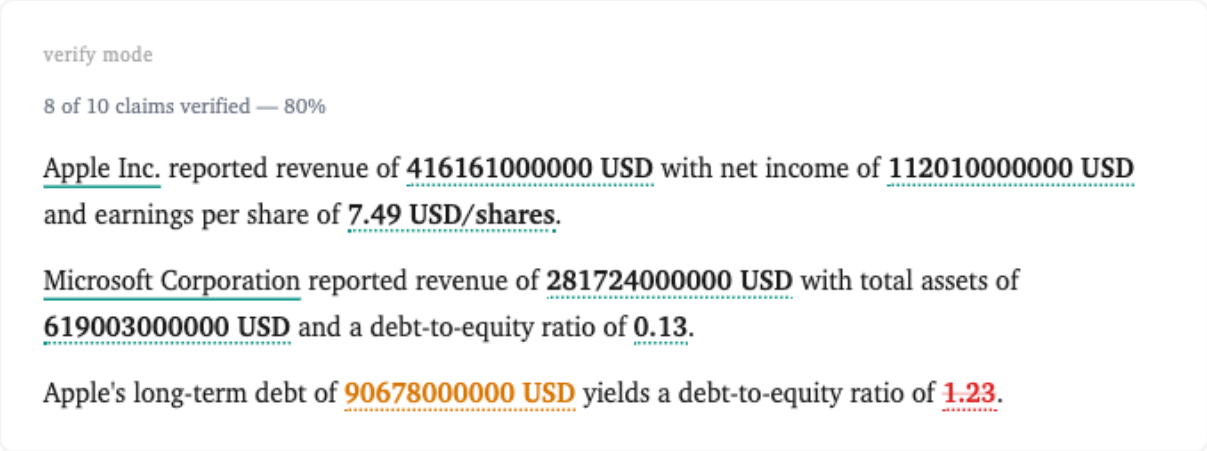


Figure 3: Verify mode on SEC EDGAR financial data. Teal dotted underlines mark verified facts; violet dashed underlines mark verified inferences. All 10 claims (8 facts, 2 inferences) pass verification.

## 7 Evaluation

We evaluated ProveML through two complementary experiments: a generation quality study measuring how well the AI produces verifiable output, and an error detection study measuring whether ProveML catches all injected errors.

verify mode — errors detected

4 of 8 claims verified — 50%

Apple Inc. reported revenue of ~~420000000000 USD~~ with net income of 112010000000 USD.

Alphabet Inc. reported assets of 619003000000 USD.

Acme Corp has 12400 EUR. The balance is critically low.

Figure 4: Verification errors. A wrong revenue figure is shown in red with strikethrough; an unknown entity (Alphabet) renders in amber; a failed inference is shown in red dashed underline.

audit mode — proof paths visible

8 of 10 claims verified — 80%

Apple Inc. [company:aapl.name] reported revenue of 416161000000 USD [company:aapl.revenue] with net income of 112010000000 USD [company:aapl.netIncome] and earnings per share of 7.49 USD/shares [company:aapl.eps] .

Microsoft Corporation [company:msft.name] reported revenue of 281724000000 USD [company:msft.revenue] with total assets of 619003000000 USD [company:msft.assets] and a debt-to-equity ratio of 0.13 [company:msft.debtToEquity] .

Apple's long-term debt of 90678000000 USD [company:msft.longTermDebt] yields a debt-to-equity ratio of 1.23 [company:msft.debtToEquity] .

Figure 5: Audit mode: proof paths visible inline, showing the exact fact store path checked for each claim (e.g., `company:aapl.revenue + ._unit`, `IS_NEGATIVE_BALANCE`).

### 7.1 Experiment 1: Generation Quality

**Setup.** We evaluated ProveML using a fully synthetic dataset inspired by an educational assessment context. The dataset contains 741 fictitious students across 95 class offerings with realistic but entirely fabricated names, scores, and class structures. No real student data was used; the dataset was constructed to be representative of typical secondary school assessment patterns without corresponding to any actual individuals or institutions. We formulated 10 representative queries spanning two roles (teacher and administrator) and 7 question categories derived from common educational reporting patterns.

**Method.** Each query was submitted to the ProveML pipeline: the LLM (Claude Opus 4.6) generated ProveML-annotated markdown, the server-side verifier checked all claims, and a correction pass was applied if errors were detected. We measured the verification rate (verified claims / total claims), the number of entities, facts, and inferences per response, and response time.

**Results.** The end-to-end pipeline (generation + verification + one correction pass where errors were detected) achieved **97.5% verification across 804 total claims** from 10 queries. Teacher-role queries (4 queries, 43 claims) averaged 99%; administrator-role queries (6 queries, 761 claims) averaged 96%. Four queries achieved 100%. Verification itself is negligible (<1ms for 394 claims); the dominant cost is LLM generation (mean 47.4s). For context, first-pass accuracy varies by model (15–93% across 3 runs, as shown in Section 7.3); this figure reflects the full verify-fix pipeline with a single strong API model (Claude Opus) and is an unrepeated demonstration, not a comparative result.

### 7.2 Experiment 2: Error Detection Rate

**Setup.** The critical question for ProveML is not whether the LLM generates correct output, but whether ProveML *detects* incorrect output. To test this, we injected 20 deliberate errors into otherwise valid ProveML text and measured whether the verifier caught each one.

**Results.** We injected 20 deliberate errors across six categories: wrong values (5), wrong entity (3), no context (2), wrong threshold (4), cross-entity swaps (2), and subtle errors including trailing spaces and swapped entity order (4). ProveML detected all 20, yielding a **100% error detection rate**. This is a consequence of deterministic equality checking: if the claimed value does not exactly match the fact store, the claim is flagged regardless of plausibility. The experiment validates that the implementation behaves as specified; robustness to malformed LLM markup is partially addressed by the convergence experiment (Section 7.3) but deserves further stress-testing.

### 7.3 Experiment 3: Capability Threshold and Error Classes

**Setup.** We ran 28 prompts across 7 categories (lookup, aggregation, comparison, threshold, overview, student, unsupported) on four models: Phi-3 Mini (3.8B), Qwen 2.5 3B, Claude Haiku (API), and Qwen 2.5 7B. Each model was evaluated 3 times; we report the mean across runs. Each query received a context-sliced data payload containing only the entities relevant to that prompt (slicing is driven by the benchmark definition, which lists the entity references each prompt needs; in a production deployment, this would be handled by a retrieval or routing layer). Up to 3 correction loops were allowed. For each model we report three metrics:

- *First-pass*: verification rate before any correction loop.
- *After loop*: verification rate after up to 3 correction passes.
- *Converged*: number of queries where every claim verified. This distinguishes models that are mostly right on every query from models that are perfect on some and broken on others.
- *Addr.*: residual addressability errors (entity-not-found, name-mismatch, field-not-found) in non-converged queries.
- *Value*: residual value errors (correct reference, wrong number) in non-converged queries.

All local models ran via Ollama on an M2 Air (16GB).

**Results.**

**Three regimes.** Across 3 runs per model, we observe a three-regime pattern:

1. **Below threshold** (Phi-3 3.8B): the model is unstable across runs (15% ± 21, range 0–44%), producing usable markup only intermittently. Addressability and context errors

| Model | First-pass | After loop | Conv. (mean) | Addr. | Value |
|---|---|---|---|---|---|
| Phi-3 Mini (3.8B) | 15% ± 21 | 15% ± 21 | 1.7/28 | 13 (65%) | 3 (15%) |
| Qwen 2.5 3B | 86% ± 1.4 | 87% ± 0.5 | 19/28 | 19 (83%) | 4 (17%) |
| Claude Haiku | 87% ± 2.8 | 88% ± 1.2 | 17.7/28 | 21 (88%) | 3 (13%) |
| Qwen 2.5 7B | 93% ± 3.6 | 93% ± 3.6 | 25/28 | 6 (75%) | 2 (25%) |

Table 3: Four-model comparison (mean ± sd over 3 runs). Addr. = addressability errors (residual, non-converged queries). Value = wrong number. Phi-3 also had context errors (missing entity binding). All models used sliced context, few-shot example, and up to 3 correction loops.

dominate. The loop provides no reliable benefit. A single lucky run appeared to show 50% verification; repeated evaluation reveals this was not stable.

2. **Middle regime** (Qwen 3B, Haiku): models produce markup reliably with 86–87% first-pass verification, stable across runs (± 1–3). Addressability errors dominate (83–88% of residuals). The loop provides modest, selective benefit (+1pp).

3. **Above threshold** (Qwen 7B): the model produces highly verifiable output on the first pass (93% ± 3.6), converging on 25 of 28 queries. ProveML's value shifts from correction to compliance: catching residual errors and providing an audit trail.

**Error classes.** Across the three stable models, residual failures are dominated by addressability errors (75–88%) rather than value mismatches. This suggests that ProveML's main benefit is not only catching wrong numbers, but exposing incorrect or malformed links between generated claims and the underlying data. Some addressability errors cascade: a single wrong entity ID can produce multiple downstream field-not-found errors.

**Context selection as a first-class concern.** All local models failed completely (0% markup) when given the full dataset as context. With sliced context, the 3B model achieved 86% first-pass verification, stable across runs. This is not a benchmark convenience: context selection (retrieval, filtering, query routing) is standard in production deployments, and ProveML should be evaluated inside that architecture. The finding is that verification and context selection work together: context selection brings models above the capability threshold, and verification ensures the claims they produce are correct. Neither alone is sufficient.

**Without ProveML, none of these verification rates are measurable.** There is no automated way to determine whether a bare-prose AI report contains correct numbers. The verify-fix loop and error classification are only possible because ProveML makes each claim addressable and checkable.

### 7.4 Experiment 4: Cross-Domain Evaluation on SEC EDGAR Data

**Setup.** To test domain-agnostic applicability, we built a second benchmark from real SEC EDGAR XBRL filings: FY2025 10-K data for Apple Inc. (CIK 0000320193, filed 2025-10-31) and Microsoft Corporation (CIK 0000789019, filed 2025-07-30). The fact store contains 10 financial fields per company (revenue, assets, liabilities, net income, EPS, cash, equity, shares outstanding, long-term debt, debt-to-equity ratio), all unit-annotated (USD, USD/shares, shares). We designed 10 prompts across 5 categories (lookup, comparison, threshold, overview, unsupported). Each model was evaluated 3 times with the same pipeline as Experiment 3: sliced context, few-shot example, up to 3 correction loops.

| Model | First-pass | After loop | Conv. (mean) |
|---|---|---|---|
| Phi-3 Mini (3.8B) | 92% ± 2.1 | 92% ± 2.1 | 8.3/10 |
| Qwen 2.5 3B | 93% ± 4.7 | 93% ± 4.7 | 9.3/10 |
| Claude Haiku | 92% ± 2.2 | 97% ± 2.4 | 8.7/10 |
| Qwen 2.5 7B | 99% ± 1.9 | 99% ± 1.9 | 9.7/10 |

Table 4: Finance benchmark (mean ± sd over 3 runs). SEC EDGAR FY2025 data, 10 prompts, 2 entities, unit-annotated. All models used sliced context, few-shot example, and up to 3 correction loops.

**Results.**

**Context complexity, not model size.** On this compact, structurally clean benchmark (2 entities, 10 fields each), all four models entered the verifiable boundary reliably — including Phi-3 Mini, which scored 92% ± 2.1 here versus 15% ± 21 on the educational benchmark (95 entities, 741 nested records). The three-regime pattern observed in Experiment 3 collapses under lower context complexity: even the weakest model produces stable, high-quality markup when the schema is compact and the entities are few.

**Loop benefit.** The verify-fix loop added almost nothing: initial and final rates are identical for three of four models. Only Haiku showed measurable loop benefit (+5pp). This confirms that with well-structured inputs, ProveML's primary value is compliance and auditability, not correction.

**Implications.** These results do not show that finance is "easier" than education; they show that the capability threshold depends strongly on benchmark structure and context complexity. The same architecture, verifier, and grammar produced very different regime patterns across domains. This validates ProveML's domain-agnostic design: the mechanism transfers; the deployment challenge is context management.

## 8 Related Work

We organise related work by what each system checks, how it checks, and where in the pipeline it operates.

**Post-hoc fact-checking (AI in the loop).** FActScore Min et al. (2023), SAFE Wei et al. (2024), FacTool Chern et al. (2023), and OpenFactCheck (COLING 2025) decompose generated text into atomic claims and verify each using LLMs or web search. ClaimVer (EMNLP 2024 Findings) adds explainability through knowledge graphs. SelfCheckGPT Manakul et al. (2023) detects hallucinations via sampling consistency without external knowledge. RefChecker Hu et al. (2024) extracts claim triplets for reference-based checking. All use AI in the verification loop, making verification itself probabilistic. ProveML's verification is deterministic.

**Attribution (where, not whether).** Anthropic's Citations API, OpenAI's response annotations, and Google's Vertex AI Grounding Google (2024) link generated text to source documents or passages. Google's grounding API additionally returns support scores and claim-to-source metadata, though verification remains probabilistic. RARR Gao et al. (2023) and WebGPT Nakano et al. (2021) attach URLs. LAQuer (ACL 2025) and "Attribute First, then Generate"

(ACL 2024) localise attributions to fine-grained source spans. These systems track *where* a claim came from, not *whether the claimed value is correct*. Verification, if any, is against unstructured text.

**Grounding guardrails (confidence scoring).** Amazon Bedrock Guardrails, Azure AI Groundedness, Vectara HHEM, Patronus Lynx, Galileo, and Ragas/FaithJudge (EMNLP 2025) produce probabilistic confidence scores for response faithfulness. These are post-generation filters, not inline markup. None performs deterministic claim-level verification against structured data. Amazon Bedrock's Automated Reasoning checks (2025) are adjacent to ProveML but not equivalent: Bedrock validates model outputs against formalized policy constraints, whereas ProveML places claim-level verification markup directly inside the generated text, binding individual claims to addressable data paths and registered threshold predicates. The overlap is strongest at the inference/policy layer; Bedrock tells you whether a response complies; ProveML lets you ship a response whose verified parts are explicitly marked, auditable, and enforceable at the level of individual claims.

**Schema validation (format, not facts).** Instructor, Outlines, LMQL, Guidance, and Pydantic AI constrain LLM output *structure*: valid JSON, correct types, matching schemas. They ensure the AI produces the right shape; ProveML ensures the AI produces the right values.

**Data-grounded generation.** The data-to-text community Parikh et al. (2020) and StructFact (ACL 2025 Findings) benchmark faithful generation from structured data. ClaimDB (arXiv, January 2026) is the first fact-verification benchmark over large structured databases, using executable SQL queries. Kamu Data's Oracle-Augmented Generation Kamu Data (2025) delegates computation to a deterministic query engine with cryptographic provenance. These approaches verify at the query level; ProveML verifies at the claim level within natural language text.

**Inline tagging.** iXBRL XBRL International (2013) embeds machine-readable tags in human-readable financial reports, the closest structural analog to ProveML. SymGen Hennigen et al. (2023) embeds symbolic references in AI-generated text that are resolved against table data. SymGen's strategy is *error prevention*: a parser substitutes placeholders with data values, so the AI never independently claims a value. ProveML's strategy is *error detection*: the AI claims values directly, and a verifier checks them (Figure 6). SymGen cannot detect mismatches or audit existing text. ProveML adds entity scoping, a threshold registry, and composable inference chains.

**Provenance standards.** W3C PROV-O Lebo et al. (2013) and C2PA Coalition for Content Provenance and Authenticity (2022) provide document-level provenance metadata. Proof-Carrying Code Necula (1997) embeds machine-checkable safety proofs in executable code. ProveML borrows the principle that the producer embeds checkable evidence; the mechanisms differ entirely.

## 9 Reference Implementation

A reference implementation is available as a markdown-it plugin (JavaScript):

- ProveML plugin (parser + renderer with lexical scope stack): 445 lines of JavaScript

| Approach | Category | Determ. | Structured | Inline | Inference |
|---|---|---|---|---|---|
| FActScore, SAFE, FacTool | Fact-checking | – | – | – | – |
| Anthropic/OpenAI/Google | Attribution | – | – | Partial | – |
| Bedrock/Azure/Lynx/HHEM | Guardrails | – | – | – | – |
| Instructor/Outlines/LMQL | Schema | ✓ | – | – | – |
| ClaimDB, Kamu OAG | Data-grounded | ✓ | ✓ | – | – |
| iXBRL* | Inline tagging | ✓ | ✓ | ✓ | – |
| SymGen | Inline tagging | ✓ | ✓ | ✓ | – |
| **ProveML** | **Inline tagging** | ✓ | ✓ | ✓ | ✓ |

Table 5: Comparison across representative verification approaches. Determ. = deterministic. Structured = against structured data. Inline = claim-level markup in text. Inference = composable threshold checks within natural language text. The Bedrock/Azure/Lynx/HHEM row reflects the predominant probabilistic grounding and guardrail mode; Amazon Bedrock's separate Automated Reasoning feature adds formal policy checks but not inline claim markup. *XBRL's formula linkbase provides document-level calculation and assertion checks; the iXBRL specification itself does not include claim-scoped inference.

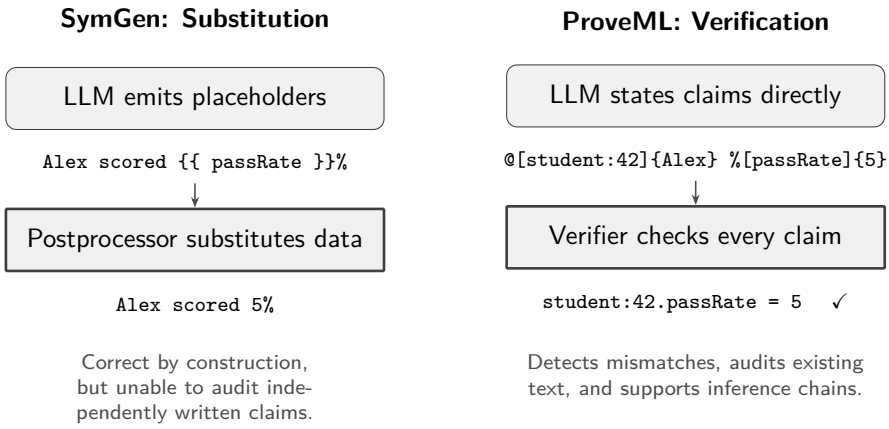


Figure 6: Architectural comparison. SymGen prevents errors by substituting placeholders with data values. ProveML detects errors by verifying independently claimed values against the data store.

- Threshold registry: 15 thresholds, extensible
- Test suite: 156 tests (78 plugin, 29 grammar, 15 paper examples, 14 CLI, 20 detection)
- Convergence harness: 28 prompts, 4 models, 3 repeats (Section 7.3)

**Fact store construction.** The server loads a JSON data file and flattens it into a key-value store at startup: each entity's fields become `type:id.field` → `value` entries. Optional `._unit` companion keys are added where units are declared. The flattening is a single pass over the source data with no external dependencies.

**Verify-fix loop.** The generation pipeline calls the LLM, passes the response to the standalone verifier, and inspects the result. If errors remain and the loop budget is not exhausted, a correction prompt is constructed containing the specific verification errors and the original data context. The corrected response is accepted only if it improves verification without dropping claims.

**Rendering integration.** The markdown-it plugin delegates verification to the same logic as the standalone verifier, then emits HTML spans with `data-*` attributes carrying the verification status, entity path, and error details. A CSS layer maps these attributes to visual indicators. The plugin and standalone verifier share the same fact store and threshold registry but are independently usable.

Tested using a fully synthetic dataset inspired by a secondary school assessment context. All data is fabricated and does not correspond to real individuals or institutions. The reference implementation is available as an npm package at https://github.com/ShaneDeconinck/proveml. The paper, benchmarks, experiment results, and reproducibility guide are at https://github.com/ShaneDeconinck/proveml-research.

# 10 Discussion

## 10.1 ProveML as Governance Infrastructure

Although ProveML can improve output quality through verify-fix loops, its deeper role is governance infrastructure: a way to place AI-generated claims inside a boundary where they can be audited, corrected, and held accountable.

The verify-fix loop is a consequence of this design. Because verification is deterministic and instant, errors can be fed back to the LLM with specific messages ("student:42.passRate: claimed 7, actual 5"), enabling targeted correction at claim-level granularity. But the loop is most useful in the middle capability regime. Without adequate context scaffolding, models may produce too little verifiable markup for the loop to help. Above threshold, verification serves primarily as an audit trail. The loop's value is in the transition zone where models are capable enough to produce markup but not reliable enough to get it right on the first pass.

The deeper value is not the loop but the boundary itself. With ProveML markup, every claim has a verification status, an addressable path, and a provenance chain. Without it, the same text is opaque: there is no automated way to determine whether a number in an AI-generated report matches the underlying data. This distinction matters for regulation, insurance, and institutional accountability.

## 10.2 Incentive-Aligned Generation

A key design goal of ProveML is that the markup language should be *easy for LLMs to use correctly*. This is not incidental; it creates a productive alignment between the model's generation objective and the system's verification objective.

**Maximising verifiable surface area.** The system prompt instructs the AI that every number must be wrapped in `%[field]{value}`, every qualitative word in `?[label: THRESHOLD]{text}`, and every entity in `@[entity:id]{name}`. This means the model is incentivised to convert as much of its output as possible into verifiable constructs. Unverified prose is the *exception*, not the norm. The more the model proves, the more verified indicators appear in the rendered output, creating a visible signal for both the user and the correction loop.

**Why LLMs are well-suited to this.** Three properties of modern LLMs make ProveML generation natural rather than burdensome:

1. **Markdown fluency**: LLMs are extensively trained on Markdown and produce it natively.

ProveML's constructs (`@[...]`, `%[...]`, `?[...]`) use characters that do not conflict with Markdown syntax, making them easy to interleave with natural language.

2. **In-context learning**: A few examples in the system prompt are sufficient for LLMs to adopt the ProveML pattern in practice. In our generation study (Section 7), prompt examples plus the full verify-fix pipeline achieved 97.5% end-to-end verification, with no fine-tuning.

3. **Structured data access**: When the fact store is provided in the context window, the model has direct access to the exact values it should reference. The task reduces to *selecting and formatting* rather than *recalling from parametric memory*, precisely the regime where LLMs are most reliable.

**Model-agnostic by design.** ProveML deliberately avoids requiring fine-tuned or specialised models. The system prompt, fact store, threshold registry, and verify-fix loop together form a *skill* that any general-purpose LLM can execute. As foundation models improve, ProveML benefits automatically: fewer first-pass errors, fewer correction iterations. The right abstraction is not "train the model to be more verifiable" but "give the agent the tools to prove its claims."

### 10.3 The Claim Boundary: Domain Experts Define What Can Be Stated

Not every sentence in AI-generated text needs to be a verified claim. The goal is not to annotate 100% of the prose, but to give domain experts a precise way to mark which claims should be checked. This creates a clear boundary between two modes of AI output:

1. **Verified claims**: facts, entity references, and inferences that resolve against the fact store and threshold registry. These are rendered with verification indicators and can be trusted to be consistent with the data.

2. **Plain prose**: text outside ProveML markup. It remains ordinary prose with no verification status and should not be treated as a checked claim.

The threshold registry defines which qualitative judgments *can* be verified. Claims inside the registry get checked deterministically. Text outside the markup remains unchecked prose. Deployments may prompt for broader use of markup, but ProveML itself does not scan or reject plain prose outside the verified layer.

In practice, the system prompt can encourage authors to keep consequential judgments inside the verified layer:

```
RULES:
- Every qualitative word MUST use ?[label: THRESHOLD]{text}
- If no threshold exists for your observation, write it as:
  "This may warrant further investigation" (NOT "This is X")
```

If the AI writes an unregistered threshold name inside a ProveML construct, the verifier returns an error. Otherwise, plain prose remains unchecked.

**Progressive adoption.** Adoption is not all-or-nothing. At Level 1 (entity references and fact values only), ProveML already detects wrong numbers. At Level 2, the threshold registry constrains qualitative language. Deploying requires only addressable data and, optionally, threshold definitions — a flat configuration file per domain. The verifier, grammar, and verify-fix loop are unchanged across domains. The design also extends naturally to settings with machine-verifiable

attestations (verifiable credentials, SSI systems, blockchain-derived state), though these applications are forward-looking and not evaluated in this paper.

### 10.4 Scope and Privacy

ProveML verifies what the AI *marks up*: every entity name, every number, and every thresholded qualitative judgment inside a ProveML construct. Text outside the markup remains unchecked prose. The system's guarantee is "consistent with the data source." The fact store is the single point of trust.

**Privacy by design.** The LLM operates on pseudonymous identifiers (`student:42`). Since ProveML syntax is deterministic and parseable, a rendering layer substitutes real names at display time; the AI never accesses personally identifiable information. The verification status travels with the markup; the PII stays behind.

### 10.5 Limitations

- **Consistency, not truth.** ProveML verifies that claims match the fact store, not that the fact store is correct. If the source data is wrong, verified claims will be wrong.
- **No unit conversion.** Units are supported as nullable metadata with exact string matching. Semantic unit equivalence (e.g., mg/dL vs mmol/L) and standardised vocabularies (UCUM, ISO 4217) are deferred to future work.
- **Evaluation scope.** The evaluation covers two domains (synthetic education, real SEC finance) across 38 prompts, 4 models, and 3 repeats. Experiment 1 is an unrepeated single-model demonstration. Whether error patterns generalise to larger schemas, more domains, or frontier models is not exhaustively established.
- **Prose outside markup.** ProveML verifies what is inside markup constructs. Text outside the markup is not checked; detecting unverified claim-bearing prose is outside the scope of the current specification.
- **Malformed input.** The tokenizer silently skips constructs with unmatched brackets or braces. Robustness to diverse malformed markup deserves further stress-testing.
- **Group claims.** Inferences evaluate against a single entity context. A claim like "5TW, 4B, and 6WE all have low coverage" cannot be verified as one inference; it must be decomposed into one inference per entity.

### 10.6 Future Work

Several extensions are deferred from the current specification: (1) standardised unit vocabularies (UCUM, ISO 4217) for semantic unit equivalence rather than string matching; (2) multiple fact stores with explicit source provenance or a source prefix on entity paths; (3) cross-implementation portability of canonicalization rules; and (5) evaluation on larger schemas, more domains, and frontier models.

## 11 Conclusion

ProveML places AI-generated claims inside a verifiable boundary. Three inline constructs link every claim to an addressable path in a data source. Verification is deterministic, with no model in the loop. An optional threshold registry extends this to qualitative judgments through composable primitives.

In our evaluation across four models (3.8B to API-scale, 3 runs each), we find that the capability threshold depends strongly on context complexity: on a 95-entity educational benchmark, Phi-3 scored 15% ± 21; on a compact SEC EDGAR finance benchmark (2 entities), the same model scored 92% ± 2.1. The architecture, verifier, and grammar were unchanged. Context management mattered more than model size.

What LLMs provide is open-ended generation. What ProveML adds is a controllable, auditable boundary around that generation. For humans, this means instant visual triage: verified claims can be trusted at a glance, and attention shifts to what is unverified. For agent loops, specific error feedback ("expected 142, got 143") enables targeted self-correction without human intervention. Both dramatically reduce verification time. The combination unlocks something that neither open-ended generation nor static verification offers alone: open-endedness with determinism.

## Declaration of Generative AI Use

Generative AI tools (Claude, GPT) were used during manuscript preparation for drafting, revision, literature search, and code review. All AI-generated content was reviewed, verified, and edited by the author. No generative AI was used to create or alter images or artwork. The author takes full responsibility for the content of the published article.

## References


L. Bainbridge. Ironies of automation. *Automatica*, 19(6):775–779, 1983.

I-C. Chern, S. Chern, S. Chen, W. Yuan, K. Feng, C. Zhou, J. He, G. Neubig, and P. Liu. FacTool: Factuality detection in generative AI – a tool augmented framework for multi-task and multi-domain scenarios. *arXiv:2307.13528*, 2023.

Coalition for Content Provenance and Authenticity. C2PA technical specification. https://c2pa.org/specifications/, 2022.

P. Elias. Google's AI chatbot Bard makes factual error in first demo. Reuters, 2023.

European Parliament. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act). Official Journal of the European Union, 2024.

S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630:625–630, 2024.

L. Gao, Z. Dai, P. Pasupat, A. Chen, A. T. Chaganty, Y. Fan, V. Zhao, N. Lao, H. Lee, D. Juan, and K. Guu. RARR: Researching and revising what language models say, using language models. In *Proc. ACL*, 2023.

Gartner. More than 80% of enterprises will have used generative AI APIs or deployed generative AI-enabled applications by 2026. Press Release, 2023.

Google. Grounding with Google search. Vertex AI documentation, https://ai.google.dev/gemini-api/docs/grounding, 2024.

Guardrails AI. Guardrails: Adding reliable AI to applications. https://github.com/guardrails-ai/guardrails, 2024.

L. T. Hennigen, S. Srivastava, S. Deng, M. Singh, and Y. Bisk. Towards verifiable text generation with symbolic references. *arXiv:2311.09188*, 2023.

HL7 International. FHIR (fast healthcare interoperability resources), release 5, 2023. URL https://hl7.org/fhir/R5/.

Xiangkun Hu, Dongyu Ru, Lin Qiu, Qipeng Guo, Tianhang Zhang, Yang Xu, Yun Luo, Pengfei Liu, Yue Zhang, and Zheng Zhang. Knowledge-centric hallucination detection. In *Proc. EMNLP*, pages 6953–6975, 2024.

Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023.

A. T. Kalai and S. S. Vempala. Calibrated language models must hallucinate. In *Proc. STOC*, 2024. arXiv:2311.14648.

A. T. Kalai, O. Nachum, S. S. Vempala, and E. Zhang. Why language models hallucinate. OpenAI, arXiv:2509.04664, 2025.

Kamu Data. Oracle-augmented generation: Connecting AI to real-time verifiable data. https://www.kamu.dev/blog/2025-01-08-oracle-augmented-generation/, 2025.

O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moberley, P. Jain, Q. Shi, and M. Zaharia. DSPy: Compiling declarative language model calls into self-improving pipelines. In *Proc. ICLR*, 2024.

T. Lebo, S. Sahoo, D. McGuinness, et al. PROV-O: The PROV ontology. W3C Recommendation, 2013.

J. D. Lee and K. A. See. Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1):50–80, 2004.

V. Magesh, F. Suber, N. Doshi, M. Cabral, A. Arriaga, and D. E. Ho. Hallucination-free? assessing the reliability of leading AI legal research tools. *Journal of Empirical Legal Studies*, 2025.

P. Manakul, A. Liusie, and M. J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proc. EMNLP*, 2023.

M. McCain, T. Millar, S. Huang, J. Eaton, K. Handa, M. Stern, et al. Measuring AI agent autonomy in practice. Anthropic Research, https://www.anthropic.com/research/measuring-agent-autonomy, 2026.

McKinsey & Company. The state of AI. https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai, 2025.

S. Min, K. Krishna, X. Lyu, M. Lewis, W. Yih, P. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proc. EMNLP*, 2023.

R. Nakano et al. WebGPT: Browser-assisted question-answering with human feedback. *arXiv:2112.09332*, 2021.

G. C. Necula. Proof-carrying code. In *Proc. POPL*, pages 106–119, 1997.

NVIDIA. NeMo Guardrails: A toolkit for programmable guardrails for LLM-based conversational applications. *arXiv:2310.10501*, 2023.

H. Orgad, M. Toker, A. Slobodkin, and Y. Belinkov. LLMs know more than they show. In *Proc. ICLR*, 2025. arXiv:2410.02707.

L. Ouyang et al. Training language models to follow instructions with human feedback. In *Proc. NeurIPS*, 2022. arXiv:2203.02155.

R. Parasuraman and V. Riley. Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39(2):230–253, 1997.

A. Parikh, X. Wang, S. Gehrmann, M. Faruqui, B. Dhingra, D. Yang, and D. Das. ToTTo: A controlled table-to-text generation dataset. In *Proc. EMNLP*, 2020.

C. Rudge. Deloitte AI report for Australian government contained fabricated citations. Fortune, 2025.

U.S. District Court, S.D.N.Y. *Mata v. Avianca, Inc.* 678 F.Supp.3d 443, 2023.

J. Wei, C. Yang, S. Wang, and L. Hou. Long-form factuality in large language models. *arXiv:2403.18802*, 2024.

XBRL International. Inline XBRL 1.1. https://www.xbrl.org/specification/inlinexbrl-part1/rec-2013-11-18/inlinexbrl-part1-rec-2013-11-18.html, 2013.

Z. Xu, S. Jain, and M. Kankanhalli. Hallucination is inevitable: An innate limitation of large language models. In *Proc. ICLR*, 2025. arXiv:2401.11817.